

Using a light source

จุดประสงค์เชิงพฤติกรรม

1. สามารถอธิบาย Ambient Light Diffuse Light and Specular Light ได้
2. สามารถเขียนโปรแกรมเพื่อแสดงแหล่งกำเนิดแสงและสร้าง Effect ให้กับวัตถุที่สร้างขึ้นมาได้

ให้นักศึกษาทำตามคำสั่งดังต่อไปนี้

1. จงอธิบาย Ambient Light Diffuse Light and Specular Light พร้อมเขียนภาพประกอบและรูปแบบการเขียนโปรแกรมสำหรับกำหนดค่าต่าง ๆ

Ambient Light



Diffuse Light

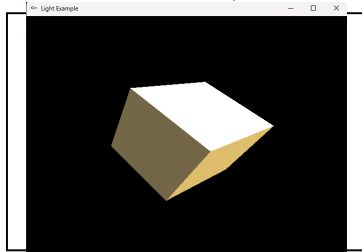


Specular Light



2. เขียนโปรแกรมสำหรับแสดงผลภาพด้วยและอธิบายคำสั่งต่าง ๆ (ตั้งชื่อไฟล์เป็นรหัสนักศึกษาLab5-1.cpp)

```
#include <windows.h>
#include <gl/glut.h>
#include <math.h>
float angle = 0.0;
void init(void)
{
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glShadeModel(GL_SMOOTH);
    glEnable(GL_DEPTH_TEST);
    GLfloat ambient[] = {0.24725, 0.1995, 0.0745, 1.0};
    GLfloat diffuse[] = {0.75164, 0.60648, 0.22648, 1.0};
    GLfloat specular[] = {0.628281, 0.555802, 0.366065, 1.0};
    GLfloat position0[] = {150.0, 150.0, 150.0, 0.0};
    GLfloat specref[] = { 0.2f, 0.2f, 0.2f, 1.0f };
    glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);
    glLightfv(GL_LIGHT0, GL_SPECULAR, specular);
    glLightfv(GL_LIGHT0, GL_POSITION, position0);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);
    glMaterialfv(GL_FRONT, GL_SPECULAR, specref);
    glMateriali(GL_FRONT, GL_SHININESS, 2);
}
void drawMyObjects()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glRotatef(angle, 1.0, 0.0, 1.0);
    angle+=0.01;
    glutSolidCube(5.0);
}
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);
    glLoadIdentity();
    gluLookAt(0.0, 0.0, 5.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);
    glScalef(0.4, 0.4, 0.4);
    drawMyObjects();
    glutSwapBuffers();
}
void reshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glFrustum(-1.0, 1.0, -1.0, 1.0, 1.5, 20.0);
    glMatrixMode(GL_MODELVIEW);
}
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(640, 480);
    glutCreateWindow("Light Example");
    init();
    glutDisplayFunc(display);
    glutIdleFunc(display);
    glutReshapeFunc(reshape);
    glutMainLoop();
    return 0;
}
```



3. ให้นักศึกษาเขียนโปรแกรมดังต่อไปนี้แล้วแสดงผลลัพธ์ (ตั้งชื่อไฟล์เป็นรหัสนักศึกษาLab5-2.cpp)

```
#include <windows.h>
#include <GL/glut.h>
#include <math.h>
static GLfloat xRot=0.0f;
static GLfloat yRot=0.0f;
void init(void)
{
    glClearColor(0.0, 0.0, 0.0, 0.5);
    glShadeModel(GL_SMOOTH);
    glEnable(GL_DEPTH_TEST);
    GLfloat ambient[] = {0.24725, 0.1995, 0.0745, 1.0};
    GLfloat diffuse[] = {0.75164, 0.60648, 0.22648, 1.0};
    GLfloat specular[] = {0.628281, 0.555802, 0.366065, 1.0};
    GLfloat position0[] = {150.0, 150.0, 150.0, 0.0};
    GLfloat specref[] = { 0.2f, 0.2f, 0.2f, 1.0f };
    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ambient);
    glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);
    glLightfv(GL_LIGHT0, GL_SPECULAR, specular);
    glLightfv(GL_LIGHT0, GL_POSITION, position0);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);
    glMaterialfv(GL_FRONT, GL_SPECULAR, specref);
    glMateriali(GL_FRONT, GL_SHININESS, 50);
}
void RenderScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    glColor3ub(255, 255, 255);
    glBegin(GL_TRIANGLES);
    glVertex3f(0.0f, 0.0f, 60.0f);
    glVertex3f(-15.0f, 0.0f, 30.0f);
    glVertex3f(15.0f, 0.0f, 30.0f);
    glColor3ub(0, 0, 0);
    glVertex3f(15.0f, 0.0f, 30.0f);
    glVertex3f(0.0f, 15.0f, 30.0f);
    glVertex3f(0.0f, 0.0f, 60.0f);
    glColor3ub(255, 0, 0);
    glVertex3f(0.0f, 0.0f, 60.0f);
    glVertex3f(0.0f, 15.0f, 30.0f);
    glVertex3f(-15.0f, 0.0f, 30.0f);
    glColor3ub(0, 255, 0);
    glVertex3f(-15.0f, 0.0f, 30.0f);
    glVertex3f(0.0f, 15.0f, 30.0f);
    glVertex3f(0.0f, 0.0f, -56.0f);
    glColor3ub(255, 255, 0);
    glVertex3f(0.0f, 0.0f, -56.0f);
    glVertex3f(0.0f, 15.0f, 30.0f);
    glVertex3f(15.0f, 0.0f, 30.0f);
    glColor3ub(0, 255, 255);
    glVertex3f(15.0f, 0.0f, 30.0f);
    glVertex3f(-15.0f, 0.0f, 30.0f);
    glVertex3f(0.0f, 0.0f, -56.0f);
    glColor3ub(128, 128, 128);
    glVertex3f(0.0f, 2.0f, 27.0f);
    glVertex3f(-60.0f, 2.0f, -8.0f);
    glVertex3f(60.0f, 2.0f, -8.0f);
```

```

glColor3ub(64,64,64);
glVertex3f(60.0f, 2.0f, -8.0f);
glVertex3f(0.0f, 7.0f, -8.0f);
glVertex3f(0.0f,2.0f,27.0f);
glColor3ub(192,192,192);
glVertex3f(60.0f, 2.0f, -8.0f);
glVertex3f(-60.0f, 2.0f, -8.0f);
glVertex3f(0.0f,7.0f,-8.0f);
glColor3ub(64,64,64);
glVertex3f(0.0f,2.0f,27.0f);
glVertex3f(0.0f, 7.0f, -8.0f);
glVertex3f(-60.0f, 2.0f, -8.0f);
glColor3ub(255,128,255);
glVertex3f(-30.0f, -0.50f, -57.0f);
glVertex3f(30.0f, -0.50f, -57.0f);
glVertex3f(0.0f,-0.50f,-40.0f);
glColor3ub(255,128,0);
glVertex3f(0.0f,-0.5f,-40.0f);
glVertex3f(30.0f, -0.5f, -57.0f);
glVertex3f(0.0f, 4.0f, -57.0f);
glColor3ub(255,128,0);
glVertex3f(0.0f, 4.0f, -57.0f);
glVertex3f(-30.0f, -0.5f, -57.0f);
glVertex3f(0.0f,-0.5f,-40.0f);
glColor3ub(255,255,255);
glVertex3f(30.0f,-0.5f,-57.0f);
glVertex3f(-30.0f, -0.5f, -57.0f);
glVertex3f(0.0f, 4.0f, -57.0f);
glColor3ub(255,0,0);
glVertex3f(0.0f,0.5f,-40.0f);
glVertex3f(3.0f, 0.5f, -57.0f);
glVertex3f(0.0f, 25.0f, -65.0f);
glColor3ub(255,0,0);
glVertex3f(0.0f, 25.0f, -65.0f);
glVertex3f(-3.0f, 0.5f, -57.0f);
glVertex3f(0.0f,0.5f,-40.0f);
glColor3ub(128,128,128);
glVertex3f(3.0f,0.5f,-57.0f);
glVertex3f(-3.0f, 0.5f, -57.0f);
glVertex3f(0.0f, 25.0f, -65.0f);
glEnd();
glPopMatrix();
glutSwapBuffers();
}
void SetupRC()
{
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_CULL_FACE);
    glFrontFace(GL_CCW);
    glClearColor(0.0f, 0.0f, 0.5f,1.0f);
}

```

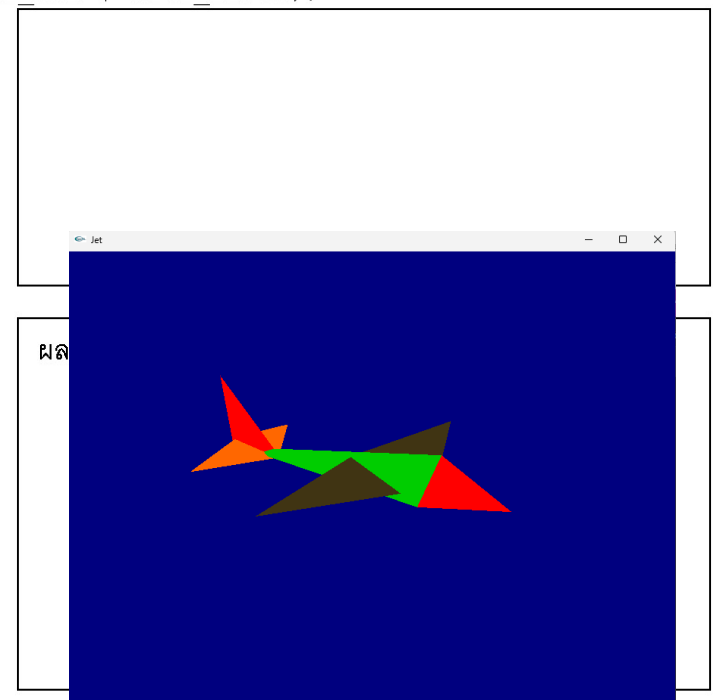
```

void SpecialKeys(int key, int x, int y)
{
    if(key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if(key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if(key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if(key == GLUT_KEY_RIGHT)
        yRot += 5.0f;
    if(key > 356.0f)
        xRot = 0.0f;
    if(key < -1.0f)
        xRot = 355.0f;
    if(key > 356.0f)
        yRot = 0.0f;
    if(key < -1.0f)
        yRot = 355.0f;
    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 80.0f;
    if(h == 0)
        h = 1;
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho (-nRange, nRange, -nRange*h/w, nRange*h/w, -nRange, nRange);
    else
        glOrtho (-nRange*w/h, nRange*w/h, -nRange, nRange, -nRange, nRange);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Jet");
    init();
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();
    return 0;
}

```



4. ให้นักศึกษาเขียนโปรแกรมโดยให้มีการปรับค่าสีตามตารางต่อไปนี้ นักศึกษาสามารถประยุกต์ใช้โปรแกรมข้อที่ 2 โดยให้นักศึกษาทำการกำหนดวัตถุเป็น กาน้ำร้อน 3 มิติ (glutSolidTeapot) ในการทดลองจากนั้นให้แสดงผลลัพธ์โดยการ Capture ภาพลงมาในไฟล์ Word แล้วตั้งชื่อไฟล์ว่า (ตั้งชื่อไฟล์เป็นรหัสนักศึกษาLab5-3.pdf)

ตารางค่าสี (Roy Hall, 1989)

Material	GL_AMBIENT	GL_DIFFUSE	GL_SPECULAR	GL_SHININESS
Brass	0.329412 0.223529 0.027451 1.0	0.780392 0.568627 0.113725 1.0	0.992157 0.941176 0.807843 1.0	27.8974
Bronze	0.2125 0.1275 0.054 1.0	0.714 0.4284 0.18144 1.0	0.393548 0.271906 0.166721 1.0	25.6
Polished Bronze	0.25 0.148 0.06475 1.0	0.4 0.2368 0.1036 1.0	0.774597 0.458561 0.200621 1.0	76.8
Chrome	0.25 0.25 0.25 1.0	0.4 0.4 0.4 1.0	0.774597 0.774597 0.774597 1.0	76.8
Copper	0.19125 0.0735 0.0225 1.0	0.7038 0.27048 0.0828 1.0	0.256777 0.137622 0.086014 1.0	12.8
Polished Copper	0.2295 0.08825 0.0275 1.0	0.5508 0.2118 0.066 1.0	0.580594 0.223257 0.0695701 1.0	51.2
Gold	0.24725 0.1995 0.0745 1.0	0.75164 0.60648 0.22648 1.0	0.628281 0.555802 0.366065 1.0	51.2
Polished Gold	0.24725 0.2245 0.0645 1.0	0.34615 0.3143 0.0903 1.0	0.797357 0.723991 0.208006 1.0	83.2
Pewter	0.105882 0.058824 0.113725 1.0	0.427451 0.470588 0.541176 1.0	0.333333 0.333333 0.521569 1.0	9.84615

ตารางค่าสี (ต่อ)

Material	GL_AMBIENT	GL_DIFFUSE	GL_SPECULAR	GL_SHININESS
Silver	0.19225 0.19225 0.19225 1.0	0.50754 0.50754 0.50754 1.0	0.508273 0.508273 0.508273 1.0	51.2
Polished Silver	0.23125 0.23125 0.23125 1.0	0.2775 0.2775 0.2775 1.0	0.773911 0.773911 0.773911 1.0	89.6
Emerald	0.0215 0.1745 0.0215 0.55	0.07568 0.61424 0.07568 0.55	0.633 0.727811 0.633 0.55	76.8
Jade	0.135 0.2225 0.1575 0.95	0.54 0.89 0.63 0.95	0.316228 0.316228 0.316228 0.95	12.8
Obsidian	0.05375 0.05 0.06625 0.82	0.18275 0.17 0.22525 0.82	0.332741 0.328634 0.346435 0.82	38.4
Pearl	0.25 0.20725 0.20725 0.922	1.0 0.829 0.829 0.922	0.296648 0.296648 0.296648 0.922	11.264
Ruby	0.1745 0.01175 0.01175 0.55	0.61424 0.04136 0.04136 0.55	0.727811 0.626959 0.626959 0.55	76.8
Turquoise	0.1 0.18725 0.1745 0.8	0.396 0.74151 0.69102 0.8	0.297254 0.30829 0.306678 0.8	12.8
Black Plastic	0.0 0.0 0.0 1.0	0.01 0.01 0.01 1.0	0.50 0.50 0.50 1.0	32
Black Rubber	0.02 0.02 0.02 1.0	0.01 0.01 0.01 1.0	0.4 0.4 0.4 1.0	10

ลายเซ็นผู้ตรวจ _____ วันที่ _____

เอกสารอ้างอิง

1. OpenGL Super Bible Third Edition, Chapter 2
2. Sumanta Guha, Computer Graphics Through OpenGL: From Theory to Experiments, 2014.
3. Roy Hall. Illumination and Color in Computer Generated Imagery. Springer-Verlag, New York, 1989. includes C code for radiosity algorithms.